

Next Route Flow: "Next" Button After Completing a Pollinator Route

Traces the complete code path from the user clicking **Next** at the end of a route through to receiving the next route — with specific focus on how the system avoids re-using routes a player has already seen.

1. Frontend — "Next" Button Click

File: `frontend/src/app/route/page.tsx`

The **Next** button is rendered inside the `RouteEndHTML` component (around line 302). Its `onButtonClick` handler calls `handleCompleteRoute()` (lines 130–170):

```
const result = await apiFetchAuthed<{
  success: boolean;
  newActiveRoute: Route | null;
  routesCompleted: number;
  message?: string;
}>("/api/complete-route", {
  method: "POST",
});
```

- **Endpoint:** `POST /api/complete-route`
- **Auth:** JWT token in `Authorization: Bearer <token>` header
- **Body:** none — all player context is derived from the JWT
- **JWT payload contains:** `{ sessionId: number; playerId: number }`

On success, the frontend receives `newActiveRoute` and navigates the player into the next route.

2. Backend Route Registration

File: `backend/src/app.ts` (line 60)

```
app.post("/api/complete-route", authenticateJWT, completeRoute);
```

The `authenticateJWT` middleware verifies the token and attaches `sessionId` and `playerId` to the request before the handler runs.

3. API Handler

File: `backend/src/api/api.ts` (lines 167–206)

```
export async function completeRoute(req, res): Promise<void> {
  const sessionId = APIUtilities.getSessionIdFromReq(req);
  const playerId = APIUtilities.getPlayerIdFromReq(req);

  const result = await gameService.completeRouteAndAdvance(sessionId, playerId);

  const newActiveRoute = result.newActiveRoute
    ? ReshapeUtilities.mapRouteDT0ToLegacyRoute(result.newActiveRoute)
    : null;

  res.status(200).json({
    success: true,
    message: "Route completed successfully",
    newActiveRoute,
    routesCompleted: result.routesCompleted,
```

```

    completedPollinators: result.completedPollinators,
  });
}

```

Extracts IDs from the JWT and hands off to `GameService` .

4. Game Service — Orchestrator

File: `backend/src/services/game.service.ts` (lines 32–55)

```

async completeRouteAndAdvance(sessionId, playerId, completionMs?) {
  // 1. Ensure a PlayerSession row exists for this (session, player) pair
  const ps = await getOrCreatePlayerSessionService.getOrCreatePlayerSession(sessionId, playerId);

  // 2. Mark the current route completed in RouteCycle
  await this.routes.completeActiveRouteForPlayerSession({
    playerSessionId: ps.playerSessionId,
    completionMs,
  });

  // 3. Fetch next route + stats in parallel
  const [routesCompleted, newActiveRoute, completedPollinators] = await Promise.all([
    this.routes.countCompletedRoutes(ps.playerSessionId),
    getNextRouteForPlayerSessionService.getNextRouteForPlayerSession({
      sessionId,
      playerSessionId: ps.playerSessionId,
      requireNodes: true,
    }),
    this.routes.getCompletedPollinators(ps.playerSessionId),
  ]);

  return { playerId: ps.playerId, routesCompleted, newActiveRoute, completedPollinators };
}

```

Three sequential things happen:

1. **Mark complete** — sets `wasCompleted = true` and `completedAt` on the active `RouteCycle` row.
2. **Select next route** — the core deduplication logic (see §5).
3. **Gather stats** — route count and pollinator list for the response.

5. Next-Route Selection Service — Core Logic

File:

`backend/src/services/db/DatabaseOperationsServices/Transactions/getNextRouteForPlayerSession.service.ts`

This is where the "never repeat a route in the same cycle" guarantee is enforced.

```

static async getNextRouteForPlayerSession(input: {
  sessionId: number;
  playerSessionId: number;
  requireNodes?: boolean;
}): Promise<RouteDTO | null> {

  let attempt = 0;
  while (true) {
    if (attempt++ >= 300) throw new Error("Failed to allocate next route after retries");

    let routeId: number | null;

    try {

```

```

routeId = await prisma.$transaction(async (tx) => {

  // A) What cycle is this player currently on?
  const currentCycle = await routeService.getLatestCycleNumber(tx, input.playerSessionId);

  // B) Try to find an unseen route in the CURRENT cycle
  let cycleNumber = currentCycle;
  let picked = await routeService.pickRandomRouteId(tx, {
    requireNodes,
    excludeSeenFor: { playerId: input.playerSessionId, cycleNumber },
  });

  // C) If no unseen routes remain, START A NEW CYCLE
  if (!picked) {
    cycleNumber = currentCycle + 1;

    // Exclude the 2 most recently completed routes to avoid immediate repeats
    const recentCompleted = await tx.routeCycle.findMany({
      where: { playerId: input.playerSessionId, wasCompleted: true },
      orderBy: { completedAt: "desc" },
      take: 2,
      select: { routeId: true },
    });
    const excludeIds = recentCompleted
      .map((r) => r.routeId)
      .filter((id): id is number => id != null);

    picked = await routeService.pickRandomRouteId(tx, {
      requireNodes,
      excludeRouteIds: excludeIds.length > 0 ? excludeIds : undefined,
    });

    // Fallback: if too few routes exist, ignore exclusions entirely
    if (!picked) {
      picked = await routeService.pickRandomRouteId(tx, { requireNodes });
    }
  }

  if (!picked) return null;

  // D) Record this assignment – creates the RouteCycle row for this route
  await tx.routeCycle.create({
    data: {
      sessionId: input.sessionId,
      playerId: input.playerSessionId,
      routeId: picked,
      cycleNumber,
      startedAt: new Date(),
      wasCompleted: false,
    },
    select: { id: true },
  });

  return picked;
});

} catch (e) {
  // Two concurrent requests raced for the same route – retry
  if (e?.code === "P2002") continue;
  throw e;
}

```

```

    if (!routeId) return null;

    // Load full DTO (route details + nodes)
    const dto = await LoadRouteDTOService.loadRouteDTO(routeId);
    if (dto) return dto;
    // Route disappeared between selection and load – retry
  }
}

```

Algorithm Summary

Phase	What Happens
A — Current cycle	Look up the highest cycleNumber in RouteCycle for this player
B — Pick unseen	Call pickRandomRouteId with excludeSeenFor; Prisma filter ensures only routes with no RouteCycle row in this cycle are candidates
C — New cycle	If nothing is left, increment cycleNumber; exclude the 2 most-recently-completed routes; pick from the full route list
Fallback	If exclusions leave zero candidates, drop them and pick any route
D — Record	Create a RouteCycle row so the route is now "seen" for this player x cycle
Concurrency	@unique([playerSessionId, routeId, cycleNumber]) on RouteCycle; P2002 violation triggers a retry (up to 300x)

6. Route Picking & Filtering

File: backend/src/services/route.service.ts (lines 65–116)

```

public async pickRandomRouteId(tx, opts): Promise<number | null> {
  const where = this.buildWhere(opts.requireNodes, opts.excludeSeenFor, opts.excludeRouteIds);

  const count = await tx.route.count({ where });
  if (count === 0) return null;

  const skip = crypto.randomInt(0, count); // uniform random offset
  const picked = await tx.route.findFirst({
    where,
    orderBy: { id: "asc" },
    skip,
    select: { id: true },
  });

  return picked?.id ?? null;
}

private buildWhere(requireNodes, excludeSeenFor?, excludeRouteIds?): Prisma.RouteWhereInput {
  const where: Prisma.RouteWhereInput = {};

  if (requireNodes) {
    where.nodes = { some: {} }; // route must have at least one node
  }

  if (excludeSeenFor) {
    // CRITICAL: Exclude routes that already have a RouteCycle row for this player+cycle
    where.cycles = {

```

```

    none: {
      playerSessionId: excludeSeenFor.playerSessionId,
      cycleNumber:     excludeSeenFor.cycleNumber,
      routeId:         { not: null },
    },
  };
}

if (excludeRouteIds?.length) {
  where.id = { notIn: excludeRouteIds }; // used when starting a new cycle
}

return where;
}

```

The `cycles: { none: {...} }` clause is the key deduplication guard. It translates to:

"Only return routes that have **no** `RouteCycle` record for this `playerSessionId` in this `cycleNumber`."

7. Database Schema — Relevant Tables

File: `backend/prisma/schema.prisma`

```

model RouteCycle {
  id          Int          @id @default(autoincrement())
  sessionId   Int
  playerSessionId Int
  routeId     Int?

  cycleNumber   Int
  startedAt     DateTime?
  completedAt   DateTime?
  completionMs  Int?
  wasCompleted  Boolean?

  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  session       Session    @relation(...)
  playerSession PlayerSession @relation(...)
  route         Route?     @relation(...)

  @@index([sessionId])
  @@index([playerSessionId])
  @@index([routeId])
  @@unique([playerSessionId, routeId, cycleNumber]) // ← prevents duplicates
}

```

One row is created per (player × route × cycle). The unique constraint is both a correctness guarantee and the mechanism that triggers concurrency retries.

8. Available Routes

Source: Database (`Route` table, seeded at startup — `backend/prisma/seed.js` / `seed.pg.js`)

`shared/data/Routes.json` has been removed. Route data is now stored exclusively in the database.

Pollinator	Nodes
Human	5

Bat	6
Hummingbird	5
Sunbird	5
Moth	6
Butterfly	6
Fly	6
Bee	7
Wasp	7
Beetle	5
Ant	4

With 11 routes, a player will see all 11 before any repeats occur. When cycle 2 begins, the 2 most recently completed routes are skipped for the first pick of the new cycle.

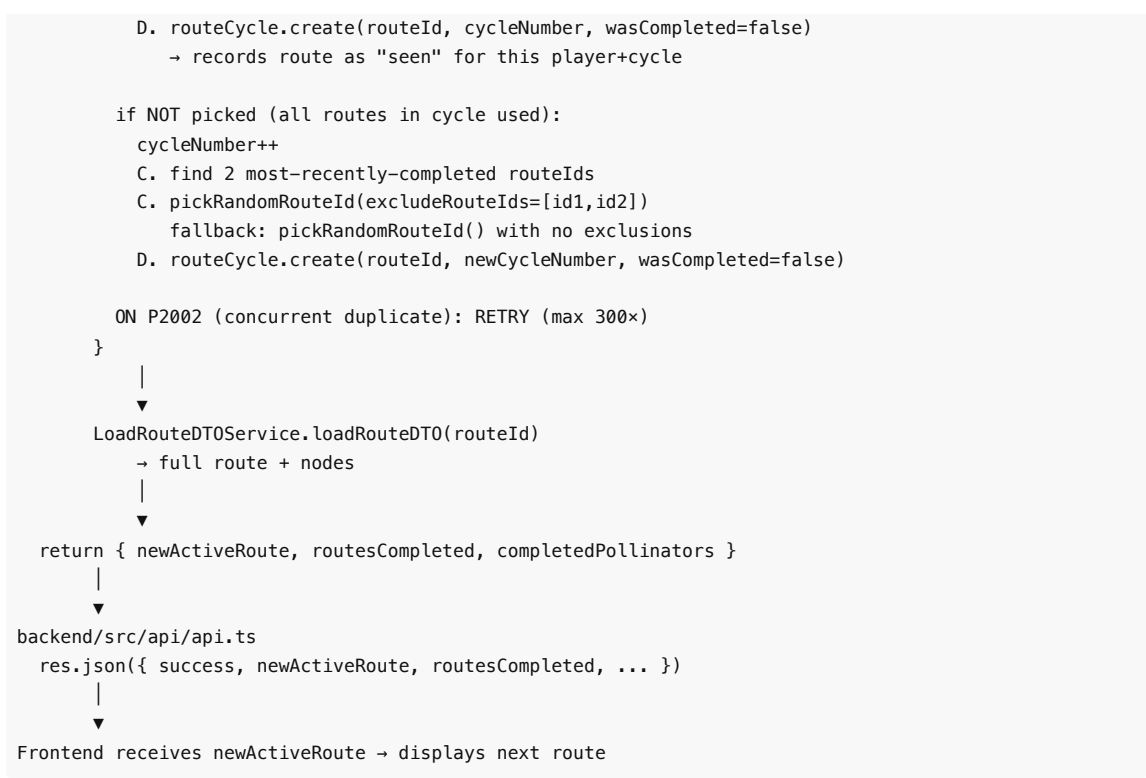
9. End-to-End Data Flow Diagram

```

USER CLICKS "NEXT"
  |
  ▼
frontend/src/app/route/page.tsx
  handleCompleteRoute()
    POST /api/complete-route (JWT: sessionId, playerId)
      |
      ▼
backend/src/app.ts
  authenticateJWT → completeRoute handler
      |
      ▼
backend/src/api/api.ts completeRoute()
  extract sessionId, playerId from JWT
      |
      ▼
backend/src/services/game.service.ts completeRouteAndAdvance()
  ┌ 1. getOrCreatePlayerSession(sessionId, playerId)
  │   → ensures PlayerSession row exists
  │
  ┌ 2. completeActiveRouteForPlayerSession(playerSessionId)
  │   → sets wasCompleted=true, completedAt=now() on current RouteCycle row
  │
  └ 3. getNextRouteForPlayerSession(sessionId, playerSessionId)
      |
      ▼
getNextRouteForPlayerSession.service.ts
TRANSACTION {
  A. getLatestCycleNumber(playerSessionId)
     → MAX(cycleNumber) from RouteCycle, default 1
  B. pickRandomRouteId(excludeSeenFor: {playerSessionId, cycleNumber})
     → route.service.ts buildWhere()
     → cycles: { none: { playerSessionId, cycleNumber } }
     → count() eligible routes
     → randomInt(0, count) → findFirst(skip=offset)

  if picked:

```



10. Potential Issues to Investigate

While the deduplication logic is correctly structured, here are edge cases worth verifying if routes are being repeated in practice:

#	Scenario	Risk
1	getLatestCycleNumber returns 1 even for a brand-new player with no RouteCycle rows	If the default is wrong, all picks land in cycle 1 and the excludeSeenFor filter works correctly — but verify the default
2	completeActiveRouteForPlayerSession does not set wasCompleted=true before getNextRouteForPlayerSession runs	The "2 most recent" exclusion at cycle boundaries depends on wasCompleted=true; if the update is delayed or skipped, the wrong routes could be excluded (or none)
3	requireNodes: true filters out routes that have no node data in the DB	If most routes lack nodes, the eligible pool is small and apparent repeats may just be the limited pool cycling fast
4	Player has multiple PlayerSession rows for the same (sessionId, playerId)	getOrCreatePlayerSession should be idempotent, but if it creates duplicate rows, the excludeSeenFor filter uses the wrong playerId
5	Concurrent POST /api/complete-route calls from the same player	The retry loop handles this, but 300 retries with 11 routes means this should never fail — check if logs show P2002 errors

Key Files Quick Reference

File	Role
frontend/src/app/route/page.tsx	"Next" button, handleCompleteRoute(), API call
backend/src/app.ts	Route registration (POST /api/complete-route)

backend/src/api/api.ts	completeRoute handler
backend/src/services/game.service.ts	completeRouteAndAdvance orchestrator
backend/src/services/db/.../getNextRouteForPlayerSession.service.ts	Core selection + deduplication logic
backend/src/services/route.service.ts	pickRandomRouteId, buildWhere (the Prisma filter)
backend/prisma/schema.prisma	RouteCycle model + unique constraint
backend/tests/unit/campbell/getNextRouteForPlayerSession.service.test.ts	Unit tests covering all branches